

Como a tecnologia funciona

SpotifyDataAnalysis: a matemática, as estruturas de dados e as decisões de engenharia por trás do recomendador e do monólito modular que o hospeda.

89.740 FAIXAS NO ÍNDICE	9 FEATURES CONTÍNUAS	$O(n)$ POR RECOMENDAÇÃO	43 REGRAS DE FRONTEIRA
---	--	--	--

.NET 8 · monólito modular (DDD + CQRS) · PostgreSQL (EF Core no write-side, Dapper no read-side) · ML.NET · React 19 + Vite + ECharts

Companheiro de `ARQUITETURA.md`, que cobre o fluxo. Aqui o assunto é o *como*.

Sumário

1. O problema, em termos numéricos

Por que "parecido" precisa de definição formal

2. Normalização: o passo sem o qual tudo desanda

Escala heterogêneas · z-score contra min-max · o piso de σ

3. Similaridade do cosseno

Direção contra magnitude · o caso degenerado · decomposição que soma o score

4. Seleção parcial: top-K sem ordenar o catálogo

Heap mínimo de capacidade fixa · $O(n \log k)$ · determinismo por desempate

5. Over-fetch adaptativo

Por que o dedup encurta a lista · janela que dobra · por que o prefixo é exato

6. Blend com sinal colaborativo

Jaccard · reescala min-max · o caso degenerado

7. Colapso de quase-duplicatas

Duas chaves independentes · por que nenhuma sozinha basta

8. A engenharia ao redor do cálculo

Mediator e pipeline · concorrência do índice · Outbox · CQRS físico

9. Como o projeto se defende de si mesmo

Regras de fronteira · gates medidos · a armadilha da vacuidade

10. Apêndice: números medidos

1. O problema, em termos numéricos

O recomendador responde a uma pergunta que parece simples: *dada uma faixa, quais são as mais parecidas?* A dificuldade é que "parecido" não tem significado até você escolher um espaço, uma métrica e uma regra de desempate. Cada escolha dessas tem consequência observável no resultado.

Cada faixa vira um ponto em $\mathbb{R}^9$. As nove coordenadas são as features contínuas de áudio: danceability, energy, valence, tempo, acousticness, instrumentalness, liveness, speechiness e loudness. A partir daí o problema é geométrico.

$$\text{faixa} \mapsto \mathbf{v} = (x_1, x_2, \dots, x_9) \in \mathbb{R}^9$$

O recomendador inteiro é uma sequência de decisões sobre como comparar dois pontos deste espaço.

O restante deste documento percorre essas decisões na ordem em que o código as aplica. Cada seção responde três perguntas: **o que faz**, **por que assim** e **o que quebra se fosse diferente**.

2. Normalização: o passo sem o qual tudo desanda

2.1 O problema das escalas heterogêneas

As nove features não vivem na mesma escala, e a diferença é brutal:

Feature	Faixa de valores	Amplitude aproximada
tempo	0 a 243 BPM	243
loudness	-50 a +5 dB	55
energy, valence, danceability, ...	0 a 1	1

Qualquer métrica que envolva produto de coordenadas fica dominada pelas duas de maior amplitude. Uma diferença de 40 BPM no tempo pesa quarenta vezes mais que a diferença máxima possível em *energy*, e nada nisso reflete percepção musical: é só o acaso da unidade de medida.

O que quebra sem normalizar O ranking vira, na prática, "faixas com BPM parecido", e as outras sete features viram ruído decorativo. O projeto tem um teste de dominância que prova os dois lados disso.

2.2 z-score, e não min-max

A transformação escolhida é a padronização z-score, aplicada coordenada a coordenada:

$$z_i = (x_i - \mu_i) / \sigma_i$$

μ e σ são a média e o desvio-padrão **populacionais** daquela feature, aprendidos numa passada única sobre o catálogo elegível.

Depois da transformação, cada coordenada tem média 0 e desvio 1. Nenhuma feature carrega mais peso por causa da unidade.

Por que não min-max Min-max ancora nos extremos: $(x - \min) / (\max - \min)$. Tempo e loudness têm cauda longa, então um único outlier comprime todo o resto do intervalo. O z-score se apoia em μ e σ , que são muito mais estáveis sob cauda. A decisão está registrada como DP-1 do épico E4.1, com o teste de dominância como evidência.

2.3 O piso de σ , e por que ele existe

Se uma feature for praticamente constante no catálogo, $\sigma \approx 0$, e dividir por ele estoura a coordenada ou produz NaN. O código trata isso explicitamente:

```
public const double MinimumStandardDeviation = 1e-9;

standardDeviations[feature] = standardDeviation < MinimumStandardDeviation
    ? 1.0 // divide por 1: a coordenada centrada colapsa para ~0
    : standardDeviation;
```

Por que 1.0 e não um epsilon Dividir por um epsilon minúsculo transformaria ruído numérico no sinal dominante do espaço. Dividir por 1 mantém a coordenada centrada perto de zero, o que é honesto: uma

feature sem variância não distingue faixa nenhuma, então ela não deve influenciar a comparação.

2.4 A invariante que sustenta tudo

Os parâmetros μ e σ são aprendidos **uma vez** e ficam fixados junto do índice. Semente e candidatos passam pela **mesma** transformação.

A falha silenciosa que isso previne Normalizar a semente com um conjunto de parâmetros e os candidatos com outro produz números plausíveis e sem sentido. É a mesma classe de erro que o skew treino/inferência causa em modelos supervisionados: nada estoura, tudo fica sutilmente errado.

3. Similaridade do cosseno

3.1 Direção, não magnitude

Com os vetores padronizados, a métrica é o cosseno do ângulo entre eles:

$$\cos(\mathbf{a}, \mathbf{b}) = (\mathbf{a} \cdot \mathbf{b}) / (\|\mathbf{a}\| \cdot \|\mathbf{b}\|) = \sum_i a_i b_i / (\sqrt{\sum a_i^2} \cdot \sqrt{\sum b_i^2})$$

Resultado em $[-1, 1]$. Vale 1 quando os vetores apontam na mesma direção.

Por que cosseno e não distância euclidiana O cosseno mede semelhança de **perfil**, ignorando magnitude. Duas faixas com o mesmo caráter sonoro, uma sistematicamente mais intensa que a outra, apontam na mesma direção e têm cosseno alto. A distância euclidiana as separaria por causa da intensidade, que não é o que se quer num recomendador content-based.

3.2 O vetor de norma zero

O cosseno é indefinido quando um dos vetores é a origem: não há direção para comparar. O código devolve **0** em vez de NaN.

Por que isso importa Um NaN num ranking envenena toda comparação que o toca, e o efeito aparece longe da causa. Devolver 0 significa "nenhuma semelhança de direção", que é uma leitura defensável e mantém a aritmética fechada. No espaço z-score isso só ocorre com uma faixa exatamente na média de todas as nove features.

3.3 A explicabilidade que não pode divergir do ranking

Aqui está, na minha leitura, a decisão mais elegante do projeto. A explicação de *por que* duas faixas se parecem não é calculada à parte: ela é a **decomposição** do próprio score.

$$c_i = (a_i \cdot b_i) / (\|\mathbf{a}\| \cdot \|\mathbf{b}\|) \Rightarrow \sum_i c_i = \cos(\mathbf{a}, \mathbf{b})$$

Cada parcela é a contribuição da i-ésima feature, e a soma reconstrói exatamente o score.

A prova é imediata: o denominador é constante em relação a i , então somar as parcelas é somar o numerador do produto escalar e dividir pelo mesmo denominador.

A propriedade que isso garante "As features que mais aproximaram" são, por construção, as de maior parcela. Como a soma reconstrói o score, **é impossível** a explicação apontar para um lado e o ranking para outro. A UI exibe as três maiores parcelas por item.

3.4 O score híbrido: cosseno mais afinidade de gênero

O ranking final do caminho content-based não é o cosseno puro. Ele soma um bônus quando candidato e semente compartilham gênero:

$$\text{score} = \cos(\mathbf{a}, \mathbf{b}) + \text{bônus}, \quad \text{bônus} = 0,05 \text{ se compartilham gênero; } 0 \text{ caso contrário}$$

A política que decide isso (`GenreAffinityPolicy`) tem três modos e uma regra de degradação honesta:

Modo	Elegibilidade	Bônus
<code>boost</code> (padrão)	todos elegíveis	+0,05 para mesmo gênero
<code>off</code>	todos elegíveis	nenhum, cosseno puro
<code>sameGenreOnly</code>	filtro duro: só mesmo gênero	não se aplica

Gênero imputado não conta Se o gênero da semente for nulo, vazio *ou imputado*, a política degrada para `off` e sinaliza `genreFallbackToCosineOnly` na resposta. Candidata com gênero imputado também não recebe bônus, mesmo que o rótulo coincida. O critério é explícito: **estimado não é medido**, e o usuário é informado quando a degradação acontece.

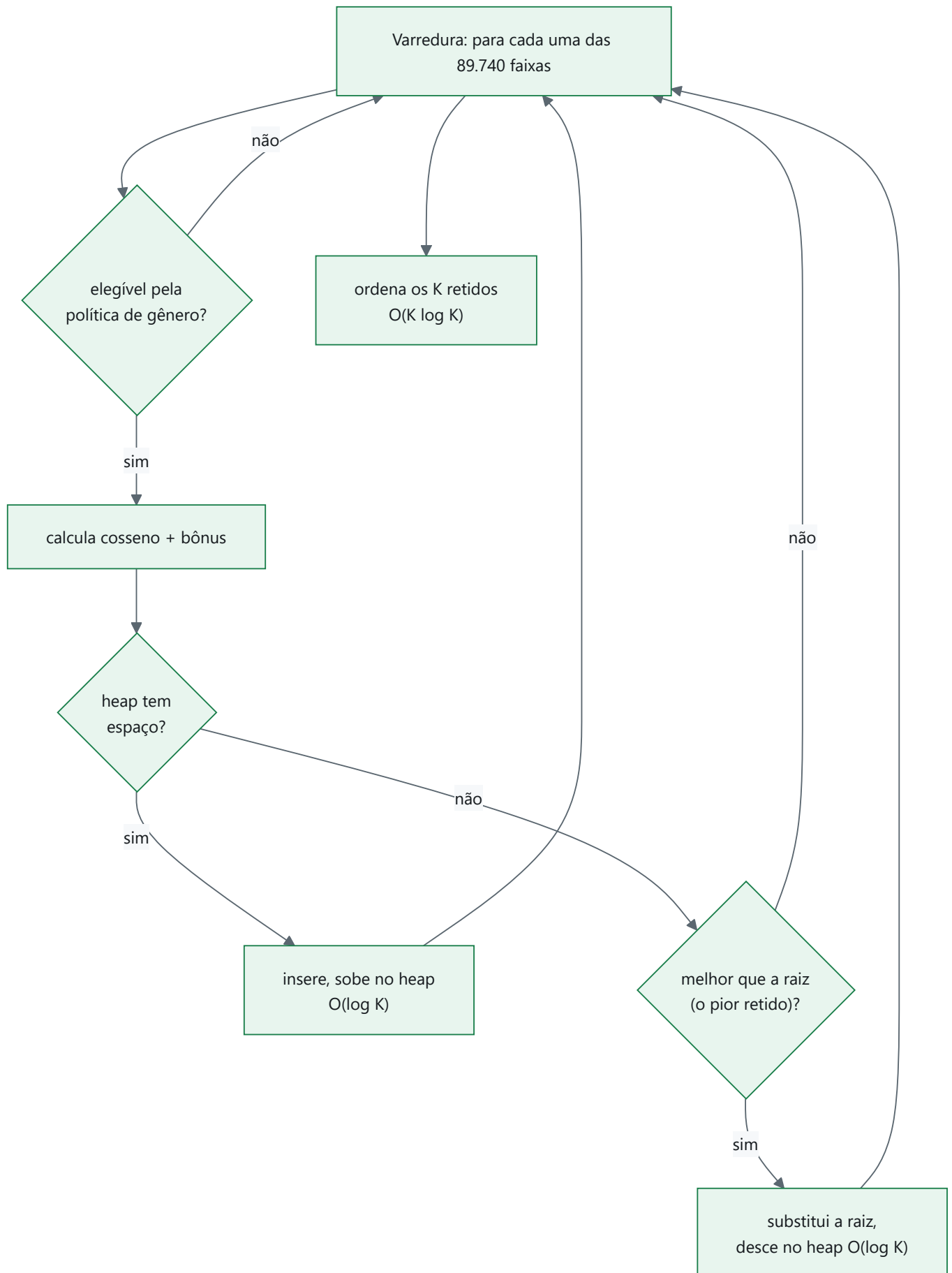
4. Seleção parcial: top-K sem ordenar o catálogo

4.1 O que seria ingênuo

Calcular o cosseno contra as 89.740 faixas e ordenar tudo custa **$O(n \log n)$** e aloca a lista inteira, para depois jogar fora 99,99% dela. Pedir 10 recomendações não justifica ordenar 89.740 elementos.

4.2 Heap mínimo de capacidade fixa

O código usa seleção parcial com um heap **mínimo** de capacidade K. A inversão é o truque: manter na raiz o *pior* dos K retidos permite decidir em **$O(1)$** se um candidato novo merece entrar.



$O(n \cdot \log K)$ em vez de $O(n \cdot \log n)$

Com $n = 89.740$ e $K = 120$, $\log_2 K \approx 7$ contra $\log_2 n \approx 16,5$. Mais importante: a memória retida é $O(K)$, não $O(n)$.

4.3 O desempate, e por que ele não é detalhe

O critério de ordem compara a similaridade e, **em caso de empate, o menor `trackId` em ordem ordinal**:

```
private static bool IsBetterThan(TrackSimilarity candidate, TrackSimilarity reference)
{
    if (candidate.Similarity > reference.Similarity) return true;
    if (candidate.Similarity < reference.Similarity) return false;
    return string.CompareOrdinal(candidate.TrackId, reference.TrackId) < 0;
}
```

Não é preferência de negócio, é repetibilidade Sem o desempate, a ordem de dois empatados dependeria da ordem em que a varredura os encontrou, que por sua vez depende da ordem de leitura do banco. O ranking deixaria de ser reprodutível e nenhuma medição de qualidade seria comparável entre execuções.

A consequência que destrava a seção 5 Como o desempate é **total** (não existem dois itens indistinguíveis), a lista ordenada é única. Logo, o *prefixo de tamanho w da lista de tamanho W é exatamente o top- w* . É esta propriedade que permite o over-fetch adaptativo sem revarrer o catálogo.

5. Over-fetch adaptativo

5.1 O problema

O usuário pede 10 recomendações. O sistema busca 10, aplica o colapso de quase-duplicatas e entrega... 4. As outras 6 eram versões da mesma faixa (remaster, ao vivo, edição deluxe) e colapsaram num representante só.

679

DE 1.895 SEMENTES RECEBIAM < 10

227

RECEBIAM EXATAMENTE 1

7,87

TAMANHO MÉDIO DA LISTA

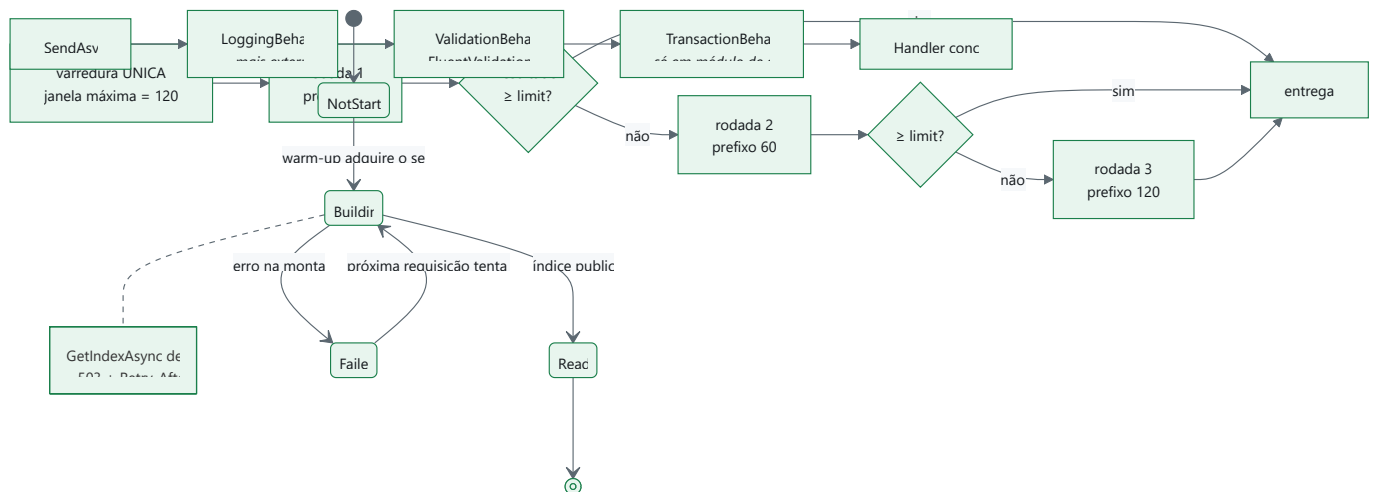
5.2 A solução, e a restrição que a molda

A saída óbvia é buscar mais candidatas do que o limite. A restrição é que a varredura é **O(n)** sobre 89.740 faixas: repetir a varredura por rodada triplicaria o custo num host de free tier.

O desenho resolve as duas coisas: **varre uma vez na janela máxima** e cada rodada pós-processa um **prefixo** dessa varredura.

$$janela(r) = \min(\max(\text{limit} \cdot 3, 10) \cdot 2^{(r-1)}, 600), \quad r \in \{1, 2, 3\}$$

Para limit = 10: 30, 60, 120. A varredura acontece uma vez, em 120.



Por que o prefixo é exato e não aproximado Pela propriedade da seção 4.3: o desempate é total, então a lista ordenada é única e o prefixo de tamanho w é o top- w que uma segunda varredura devolveria. O reaproveitamento não troca precisão por velocidade; ele é matematicamente equivalente.

5.3 A janela corta as duas fontes

No modo blend existem duas listas de candidatas: as vizinhas de áudio e as co-ocorrentes. A janela precisa cortar **ambas**.

O bug que isso corrigiu Enquanto só a lista de áudio era recortada, a rodada 1 disputava com o top-120 colaborativo em vez do top-30. Como a pior candidata de áudio da janela recebe score normalizado 0 (ver

seção 6.2), qualquer colaborativa extra com Jaccard positivo a deslocava do top-10. O resultado mudava para sementes que já estavam saudáveis, sem nenhuma rodada extra ter ocorrido. Corrigido no commit `b226f7a`.

5.4 O resultado medido

Indicador, sobre as mesmas 1.895 sementes	Antes	Depois
sementes com menos de 10 recomendações	679 (35,8%)	0
sementes com exatamente 1 recomendação	227	0
tamanho médio da lista	7,87	10,00

Distribuição de esforço: **64,2%** das sementes resolvem na rodada 1, **29,7%** na rodada 2, **6,2%** na rodada 3. Nenhuma bate no teto sem resolver. O custo é p50 +1,8 ms e média +2,8 ms; a variação no p95 fica dentro do ruído.

6. Blend com sinal colaborativo

6.1 Um segundo sinal, de natureza diferente

Além da similaridade de áudio, existe o sinal colaborativo: duas faixas que aparecem juntas em muitas playlists provavelmente combinam, mesmo que sonoramente sejam distintas. A medida é o **índice de Jaccard**:

$$J(A, B) = |P_A \cap P_B| / |P_A \cup P_B|$$

P_X é o conjunto de playlists que contêm a faixa X. Resultado em $[0, 1]$.

A matriz é pré-computada (1.481.511 pares, 254 MB) e guardada como par canônico `track_id_low / track_id_high`, o que corta o armazenamento pela metade ao custo de a consulta precisar procurar a semente nos dois lados.

6.2 Combinar duas escalas incomparáveis

O problema: o score de conteúdo vive aproximadamente em $[-1, 1]$ mais o bônus, e o Jaccard vive em $[0, 1]$, com distribuição completamente diferente. Somar direto faria uma das parcelas dominar por magnitude, não por mérito.

A solução é reescalar a parcela de conteúdo por min-max **dentro do conjunto de candidatas daquela consulta**, e só então combinar:

$$\text{contentNorm} = (\text{score} - \text{min}) / (\text{max} - \text{min})$$

$$\text{final} = (1 - w) \cdot \text{contentNorm} + w \cdot J$$

w é o `blendWeight`, padrão 0,35.

Por que min-max aqui, se a seção 2 rejeitou min-max São problemas diferentes. Na seção 2, min-max seria aplicado ao espaço global de features, onde outliers de cauda longa o tornariam instável. Aqui ele é aplicado a um conjunto pequeno e já filtrado (as candidatas de uma consulta), e o objetivo não é preservar distribuição: é só trazer duas escalas para a mesma faixa antes de combiná-las. A transformação é afim e **monótona**, então a ordem relativa dentro da parcela de conteúdo é preservada, inclusive o efeito do bônus de gênero.

6.3 O caso degenerado

Se todas as candidatas tiverem o mesmo score de conteúdo, $\text{max} - \text{min} = 0$ e a reescala dividiria por zero. O código trata explicitamente, devolvendo 0 ou 1 conforme o caso, em vez de propagar `NaN`.

6.4 O score exposto é o score que ordenou

O bug que definiu esta regra O blend reordenava pelo cosseno *reescalado* em vez do score híbrido. O bônus de gênero era calculado e depois descartado na ordenação. O efeito medido foi de **-5,15σ** na coerência de gênero, e passou dois épicos despercebido porque o próprio instrumento de medição continha o mesmo erro. Hoje o contrato é explícito: existe **um número só**, que ordena e que aparece na resposta.

Consequência prática para quem consome a API, e que já causou uma tela mentindo:

Estratégia	score significa	A soma cosine + genre fecha?
content	coseno + bônus de gênero	sim
blend	score final do blender, normalizado	não

7. Colapso de quase-duplicatas

O catálogo tem a mesma música várias vezes: remaster, ao vivo, edição deluxe, single e álbum. Entregar as cinco como "recomendações distintas" é inútil. O colapso usa **duas chaves independentes**, e nenhuma das duas sozinha resolve.

7.1 Chave numérica: cosseno quase perfeito

mesma faixa se $\cos(a, b) \geq 0,999$

Limiar deliberadamente severo: é "features praticamente idênticas", não "parecidas".

Pega o caso do mesmo master distribuído em álbuns diferentes, onde as features são bit a bit iguais.

7.2 Chave textual: nome normalizado

A segunda chave normaliza artista e título de forma agressiva: corta sufixo após `-`, remove conteúdo entre parênteses e colchetes, tira acentos e pontuação, colapsa espaços, e monta `"artista|título"`.

"Creep - Radio Edit"	→ "radiohead creep"
"Creep (Acoustic Version)"	→ "radiohead creep"
"Creep [2008 Remaster]"	→ "radiohead creep"

Por que as duas chaves O cosseno sozinho perde o remaster cujas features foram medidas de novo e diferem na terceira casa. A chave textual sozinha perde a mesma gravação publicada com títulos diferentes, e agrupa faixas homônimas de artistas distintos se a normalização errar. Juntas, cobrem os dois eixos de falha.

7.3 A ordem importa

O colapsador abre grupos **enquanto** o número de grupos for menor que o limite, percorrendo as candidatas na ordem do ranking. Não é um pós-filtro. Isso preserva a posição: o representante fica no lugar da melhor versão do grupo, e o contador `equivalentVersionsCollapsed` informa quantas foram absorvidas.

8. A engenharia ao redor do cálculo

8.1 CQRS com mediator próprio

O projeto não usa MediatR. O mediator é caseiro: resolve o handler por reflexão, com cache em `ConcurrentDictionary<Type, MethodInfo>`, e envolve a chamada nos behaviors registrados.

A ordem é contraintuitiva O primeiro behavior registrado vira o mais externo. Como `LoggingBehavior` é registrado antes, log envolve validação, e uma requisição que falha na validação *aparece* no log. O `CLAUDE.md` documenta a ordem invertida; o código é a fonte da verdade.

8.2 Concorrência: servir um índice que ainda não existe

O índice é singleton, custa uma varredura completa para montar, e é montado em background no arranque. Isso cria uma janela em que o processo aceita HTTP mas não consegue responder. O desenho torna esse estado **explícito** em vez de inferido.

Três detalhes de concorrência que valem estudo:

1. **A ordem de publicação.** `_index` é escrito *antes* de `_state = Ready`. Com ambos `volatile`, a escrita volátil tem semântica de *release*: quem lê `Ready` tem garantia de enxergar o índice. A ordem inversa criaria uma janela com estado pronto e referência nula.
2. **O estado não é inferido do semáforo.** A versão anterior deduzia "indisponível" de `_gate.CurrentCount == 0`. Isso é detalhe de implementação do `SemaphoreSlim`, e criava duas falhas: um 503 lançado depois de o índice já ter ficado pronto, e uma requisição chegando antes do warm-up que *pagava a varredura inteira de forma síncrona* com o warm-up enfileirado atrás dela.
3. **A sinalização acontece no construtor.** O hosted service avisa o provider no construtor, não no `ExecuteAsync`. O host resolve todos os `IHostedService` antes de subir o Kestrel, então quando a primeira requisição chega, o provider já sabe que um warm-up vem a caminho e devolve 503 em vez de construir sob demanda.

8.3 CQRS levado à separação física

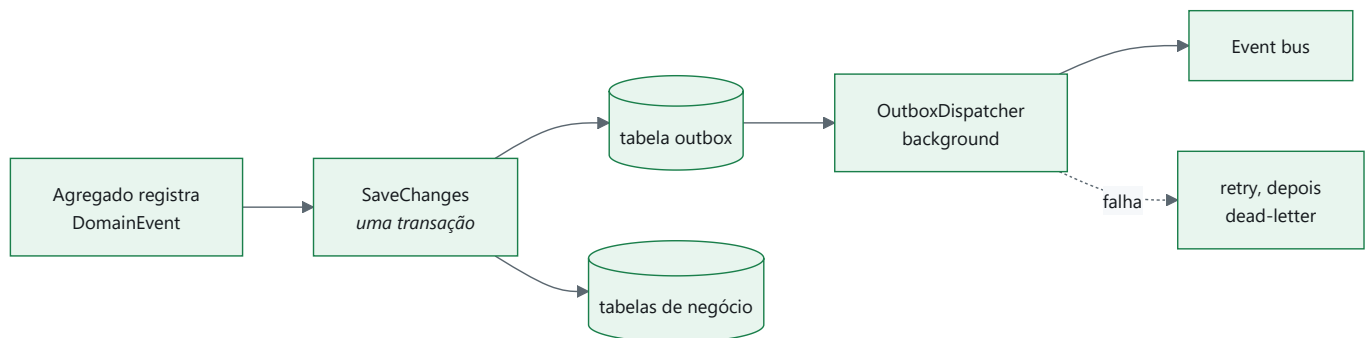
A separação não é só nominal, é de tecnologia de acesso:

	Write-side	Read-side
Tipo	<code>ICommand</code>	<code>IQuery</code>
Acesso	EF Core , rastreamento e unidade de trabalho	Dapper , SQL escrito à mão
Organização	agregado no Domain, handler no Application	Query, Handler e Result no mesmo arquivo
Transação	<code>TransactionBehavior</code>	nenhuma
Paginação	não se aplica	<code>ROW_NUMBER() OVER()</code> + <code>COUNT(*) OVER()</code> numa CTE

A armadilha do `COUNT` `COUNT(*) OVER()` do PostgreSQL devolve `bigint`, que materializa como `long`. Tipar como `int` compila, passa nos testes de contrato (que afirmam apenas o texto do SQL) e produz HTTP 500 em 100% das chamadas. Custou um endpoint inteiro quebrado com 18 testes verdes.

8.4 Outbox: entrega sem perder e sem travar

Efeito colateral entre módulos não roda dentro da transação de negócio. O evento é gravado numa tabela outbox **na mesma transação** que a mudança de estado, e um job em background publica depois.



A garantia que isso compra Ou a mudança de estado e o registro do evento acontecem juntos, ou nenhum dos dois. Não existe o caso "salvou mas não avisou". E falha na publicação não derruba a operação principal: ela vira nova tentativa. A contrapartida é entrega *ao menos uma vez*, então o consumidor precisa ser idempotente.

9. Como o projeto se defende de si mesmo

9.1 Fronteiras verificadas pelo compilador e pelo teste

São 43 regras ArchUnitNET que falham o `dotnet test` no instante em que uma fronteira é violada: Domain de um módulo tocando outro módulo, Domain referenciando EF Core ou ASP.NET, SharedKernel referenciando infraestrutura, Host referenciando Domain interno.

9.2 A armadilha da vacuidade

Quatro regras que não testavam nada Durante vários épicos, quatro das regras avaliavam **zero tipos**. `ResideInAssembly(string)` compara com o `FullName` do assembly, e recebia o nome simples: filtro vazio. Uma regra sobre conjunto vazio passa sempre. Com `WithoutRequiringPositiveResults()` por cima, o verde era silencioso. O que segurava a pureza do SharedKernel era o `.csproj` sem referências, não o teste. Existe hoje uma regra cuja única função é impedir a reincidência: ela verifica que os assemblies foram efetivamente descobertos. E ficou o protocolo: **regra nova só conta como provada depois de ser vista vermelha com uma violação injetada de propósito**.

9.3 Gates numéricos sobre o recomendador

Nove limiares são medidos contra o catálogo real e falham o build se regredirem:

Limiar	Régua	Medido
coerência de gênero	$\geq 0,40$	0,5563
ganho sobre cosseno puro	$\geq 2,0\times$	4,88×
saturação de gênero	$\leq 0,50$	0,2800
recall de duplicatas	$\geq 0,45$	0,6464
alcance de duplicatas	$\geq 0,50$	0,7037
cosseno médio entre irmãs	$\geq 0,98$	0,999028
repetição no top-N pós-dedup	$= 0$	0 de 1.895
taxa de lista curta	$\leq 0,05$	0,0000
sementes com um item só	$= 0$	0

Um limiar só vale na população que o calibrou A taxa de lista curta foi calibrada sobre os 111 grupos de 11 ou mais quase-duplicatas, onde o fenômeno é real (35,8%). Numa amostra sorteada do catálogo ele é praticamente zero, então o mesmo limiar de 5% seria satisfeito trivialmente. Por isso a medição carrega **identidade de população**, e o gate recusa medição que não venha da população de calibração. Sem isso, trocar a amostra deixaria o gate verde mesmo revertendo a correção que ele existe para proteger.

10. Apêndice: números medidos

CATÁLOGO E ÍNDICE

faixas no catálogo	89.740
faixas com features imputadas	0
artistas distintos	29.811
gêneros distintos	113
dimensão do espaço de similaridade	9
tempo de montagem do índice	1,05 a 1,68 s
memória retida pelo índice	39,7 a 40,8 MB

MATRIZ DE CO-OCORRÊNCIA

pares armazenados	1.481.511
ocupação em disco	254 MB de 388 MB do banco
cobertura do catálogo	9.664 faixas (10,8%)

LATÊNCIA POR RECOMENDAÇÃO, ENDPOINT LOCAL EM RELEASE

Percentil	content	observação
p50	10,60 a 11,71 ms	a varredura é ~96% do tempo
p95	14,15 a 15,07 ms	
p99	16,15 a 16,50 ms	

PEGADA EM CONTAINER

Tier	RAM após 1.800 requisições	OOM
512 MB, sem limite de CPU	310,9 MiB	não
256 MB, sem limite de CPU	148,3 MiB	não

O que estes números não dizem A medição em container rodou **sem limite de CPU**, e o gargalo real do free tier é CPU, não RAM. Latência e cold start em container ainda não foram medidos com metodologia confiável, e por isso não aparecem aqui.